# CS-317 OPERATING SYSTEMS

Lecture 23

# File management

Book 1 Chapter 12

# In the last lecture…

- We discussed B-trees.
- We explored file directories.
- We discussed file sharing.
- We understood record blocking.

# Secondary Storage Management

All about disks

# Issues with storing files on secondary storage

- On secondary storage, a file consists of a collection of blocks.
- The operating system or file management system is responsible for allocating blocks to files.
- This raises two management issues:
  - Allocation of disk space to files, and
  - Keeping track of free space.
- These two tasks are related; that is, the approach taken for file allocation may influence the approach taken for free space management.
- There is an interaction between file structure and allocation policy.

# File allocation issues

- Space is allocated to a file as one or more contiguous units, referred to as portions. A portion is a contiguous set of blocks. The size of a portion can range from a single block to the entire file.
- The issues involved in file allocation are:
  1. When a new file is created, is the maximum space required for the file allocated at once?
  2. What size of portion should be used for file allocation?
  3. What sort of data structure or table is used to keep track of the portions assigned to a file?
- To keep track of allocated portions, DOS and some other systems use a file allocation table (FAT).

# Preallocation versus dynamic allocation

- A preallocation policy requires that the maximum size of a file be declared at the time of the file creation request.
- In a number of cases, such as program compilations, the production of summary data files, or the transfer of a file from another system over a communications network, this value can be reliably estimated.
- However, for many applications, it is difficult if not impossible to estimate reliably the maximum size of the file.
- In those cases, users and application programmers tend to overestimate file size so as not to run out of space.
- This results in wastage of space on secondary storage.
- It is advantageous to use dynamic allocation, which allocates space to a file in portions as needed.

# Portion size

- At one extreme, a portion large enough to hold the entire file is allocated.
- At the other extreme, space on the disk is allocated one block at a time.
- In choosing a portion size, there are four points to consider:
  1. Contiguity of space increases performance.
  2. Having a large number of small portions increases the size of tables needed to manage the allocation information.
  3. Having fixed-size portions (e.g., blocks) simplifies space reallocation.
  4. Having variable-sized or small fixed-size portions minimizes waste of unused storage due to overallocation.

# Alternatives for decision of portion size

- Variable, large contiguous portions:
  - This provides better performance due to contiguity.
  - The variable size avoids waste.
  - The file allocation tables are small.
  - Space is hard to reuse.
- Blocks:
  - Small fixed portions provide greater flexibility in space reuse.
  - They require large tables or complex structures for allocation.
  - We do not strive for contiguity; blocks are allocated as needed.

When a file uses variable-size contiguous blocks, it is difficult to use leftover gaps on disk when files are deleted or shrunk

# Preallocation

- In the case of variable, large contiguous portions,
  - A file is preallocated one contiguous group of blocks.
  - This eliminates the need for a file allocation table; all that is required is a pointer to the first block and the number of blocks allocated.
- In the case of blocks,
  - All of the portions required are allocated at one time.
  - This means that the file allocation table for the file will remain of fixed size, because the number of blocks allocated is fixed.

# Fragmentation of free space

free space :
It is the empty space on a disk that is not currently occupied by any file.

- Free space is fragmented when it is not contiguous.
- With variable-size portions, we need to find a free space large enough to accommodate our file.
- The following are possible strategies:
  - First fit: Choose the first unused contiguous group of blocks of sufficient size from a free block list.

Consider you want to store a file of 4KB.

Your free space list is this: 2KB, 5KB, 3KB, 6KB, 4.5KB, & 7 KB.

First fit provides you the second free space of 5KB.

The free space list after allocation is: 2KB, 1KB, 3KB, 6KB, 4.5KB, & 7KB.

# Fragmentation of free space

- Free space is fragmented when it is not contiguous.
- With variable-size portions, we need to find a free space large enough to accommodate our file.
- The following are possible strategies:
  - Best fit: Choose the smallest unused group that is of sufficient size.

Consider you want to store a file of 4KB.

Your free space list is this: 2KB, 5KB, 3KB, 6KB, 4.5KB, & 7 KB.

Best fit provides you the second last free space of 4.5KB.

The free space list after allocation is: 2KB, 5KB, 3KB, 6KB, 0.5KB, & 7KB.

# Fragmentation of free space

- Free space is fragmented when it is not contiguous.
- With variable-size portions, we need to find a free space large enough to accommodate our file.
- The following are possible strategies:
  - First fit: Choose the first unused contiguous group of blocks of sufficient size from a free block list.
  - Best fit: Choose the smallest unused group that is of sufficient size.
  - Nearest fit: Choose the unused group of sufficient size that is closest to the previous allocation for the file to increase locality.

we start scanning towards the right from the prev allocation, will work like the first fit from that scan

# Pause the video!

Write your answer in a word file. Save it as <my_roll_no>_lecture_23. If your roll no is 500, the file will be named 500_lecture_23. Do not add <> signs to the filename.

## Task# 1: Why do we dislike fragmented free space?

Fragmented free space is undesirable because it breaks available memory or disk space into small, scattered pieces.`

## Task# 2: What is the cause of fragmentation of free space?

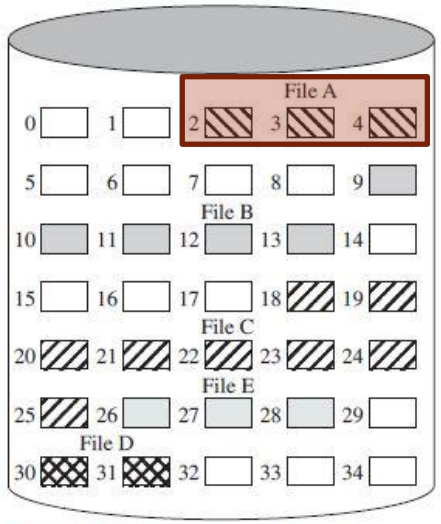
Fragmentation of free space occurs due to repeated allocation and deallocation of variable-sized files over time.

# File allocation methods

- Three file allocation methods are in common use:
  - Contiguous,
  - Chained and
  - Indexed.

# Contiguous allocation

- With contiguous allocation, a single contiguous set of blocks is allocated to a file at the time of file creation.
- This is a preallocation strategy, using variable-sized portions (a portion can have any number of blocks).
- The file allocation table needs just a single entry for each file, showing the starting block and the length of the file.
- Contiguous allocation is the best from the point of view of the individual sequential file. Multiple blocks can be read in at a time to improve I/O performance for sequential processing.
- It is easy to retrieve a single block. If a file starts at block b, and the $i^{th}$ block of the file is wanted, its location on secondary storage is simply $b + i - 1$.



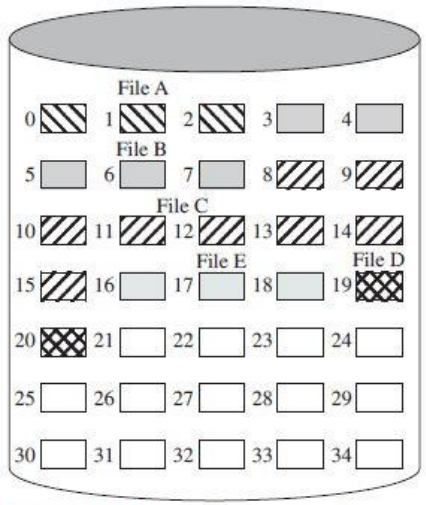

File allocation table

| File name | Start block | Length |
|---|---|---|
| File A | 2 | 3 |
| File B | 9 | 5 |
| File C | 18 | 8 |
| File D | 30 | 2 |
| File E | 26 | 3 |

Figure 12.9 Contiguous File Allocation

# Consequences of contiguous allocation

- With preallocation, it is necessary to declare the size of the file at the time of creation.
- External fragmentation is the situation in which there are lots of small free spaces in storage.
- External fragmentation will occur, making it difficult to find contiguous blocks of space of sufficient length.
- From time to time, it will be necessary to perform a compaction algorithm to free up additional space on the disk.




File allocation table

| File name | Start block | Length |
|---|---|---|
| File A | 0 | 3 |
| File B | 3 | 5 |
| File C | 8 | 8 |
| File D | 19 | 2 |
| File E | 16 | 3 |

Figure 12.10 Contiguous File Allocation (After Compaction)

# Chained allocation

- Allocation is on an individual block basis. Each block contains a pointer to the next block in the chain.
- The file allocation table needs just a single entry for each file, showing the starting block and the length of the file.
- Although preallocation is possible, it is more common to allocate blocks as needed. Any free block can be added to a chain.
- There is no external fragmentation to worry about because only one block at a time is needed.
- To select an individual block of a file requires tracing through the chain to the desired block.

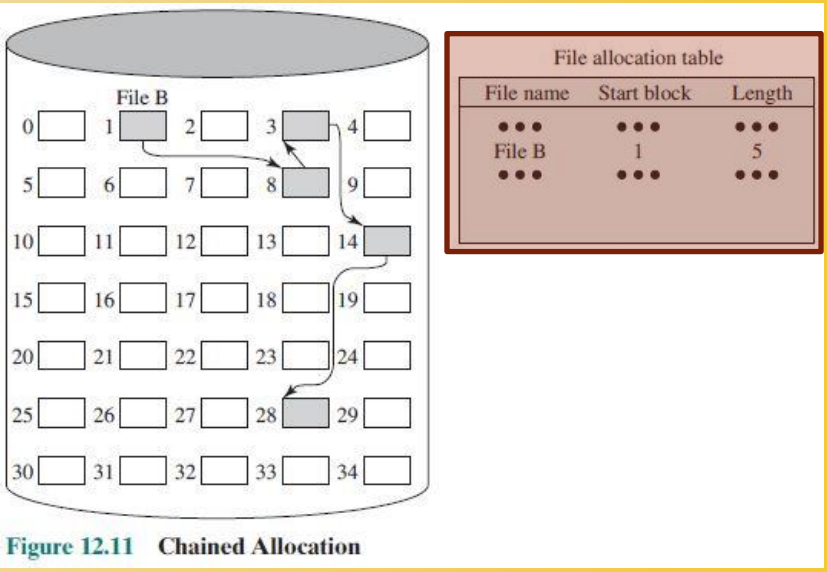


Figure 12.11 Chained Allocation

# Consequences of chained allocation

- There is no accommodation of the principle of locality. The file is scattered and fragmented.
- If it is necessary to bring in several blocks of a file at a time, a series of accesses to different parts of the disk are required.
- This affects performance more in a single-user system, but may also be of concern on a shared system.
- To overcome this problem, some systems periodically consolidate files.

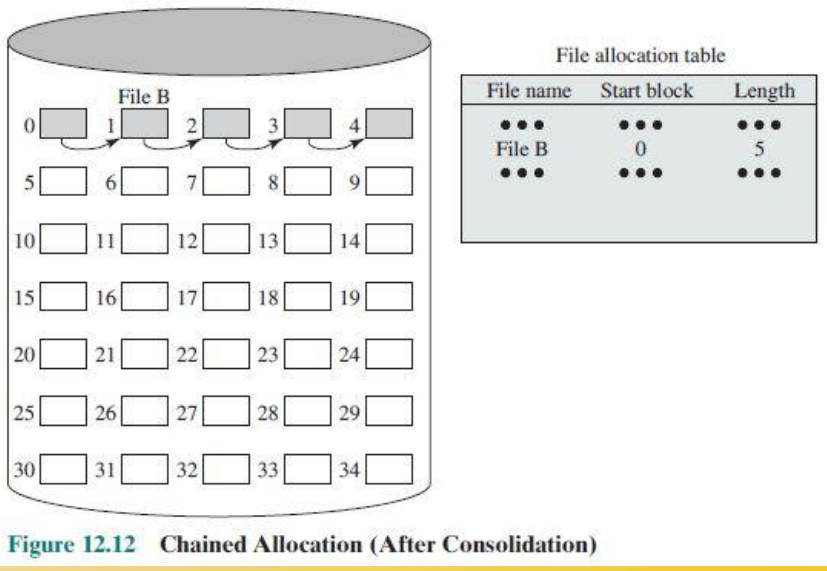


Figure 12.12 Chained Allocation (After Consolidation)

File consolidation means rearranging and combining scattered file blocks on disk to create continuous free space and reduce fragmentation

# Indexed allocation

- Indexed allocation addresses many of the problems of contiguous and chained allocation.
- Allocation may be on the basis of either fixed-size blocks or variable-sized portions.
- The file allocation table contains a separate one-level index for each file; the index has one entry for each portion allocated to the file.
- Typically, the file indexes are not physically stored as part of the file allocation table.
- Rather, the file index for a file is kept in a separate block, and the entry for the file in the file allocation table points to that block.

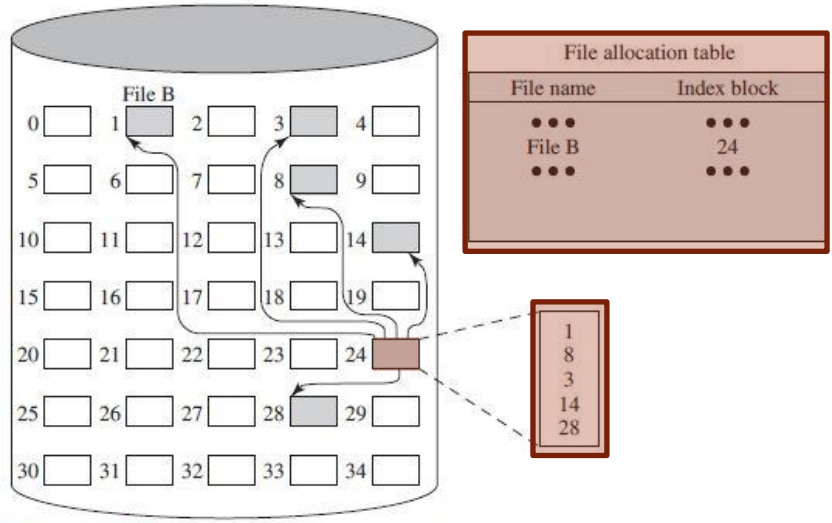



Figure 12.13 Indexed Allocation with Block Portions

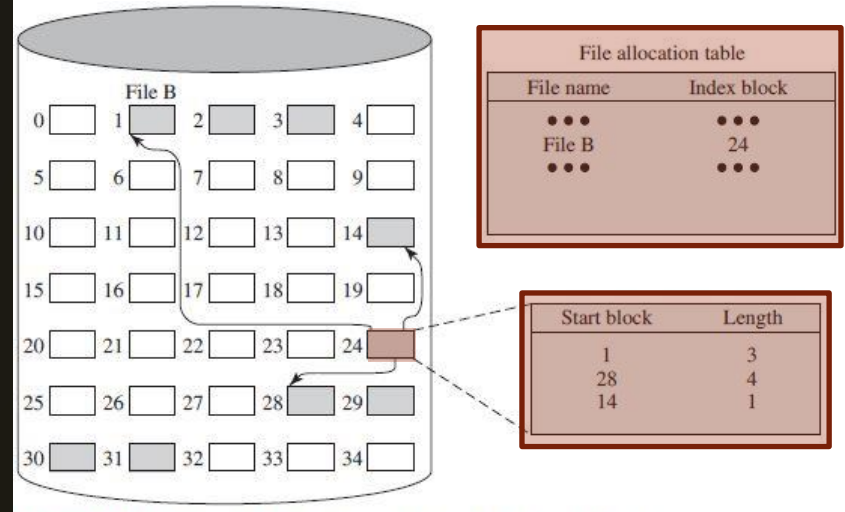



Figure 12.14 Indexed Allocation with Variable-Length Portions

# Consequences of indexed allocation

- Allocation by blocks eliminates external fragmentation.
- Allocation by variable-sized portions improves locality.
- In either case, file consolidation may be done from time to time.
- File consolidation reduces the size of the index in the case of variable-sized portions.
- Indexed allocation supports both sequential and direct access to the file and thus is the most popular form of file allocation.

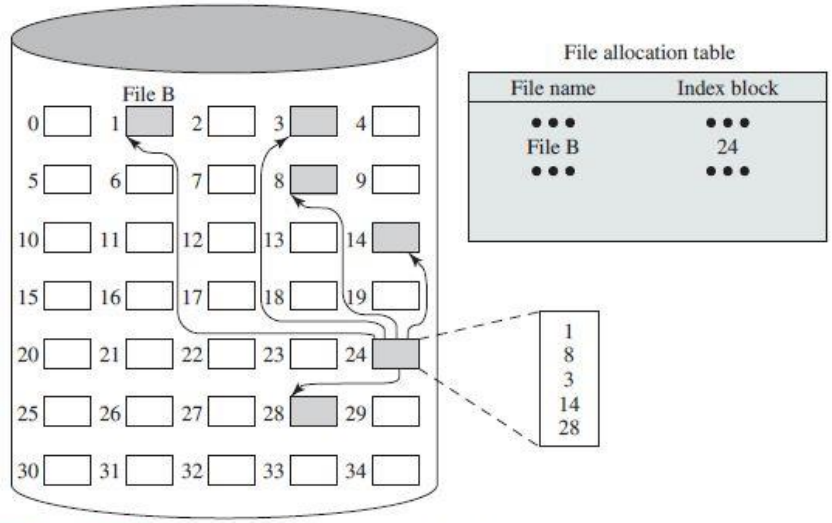



Figure 12.13 Indexed Allocation with Block Portions

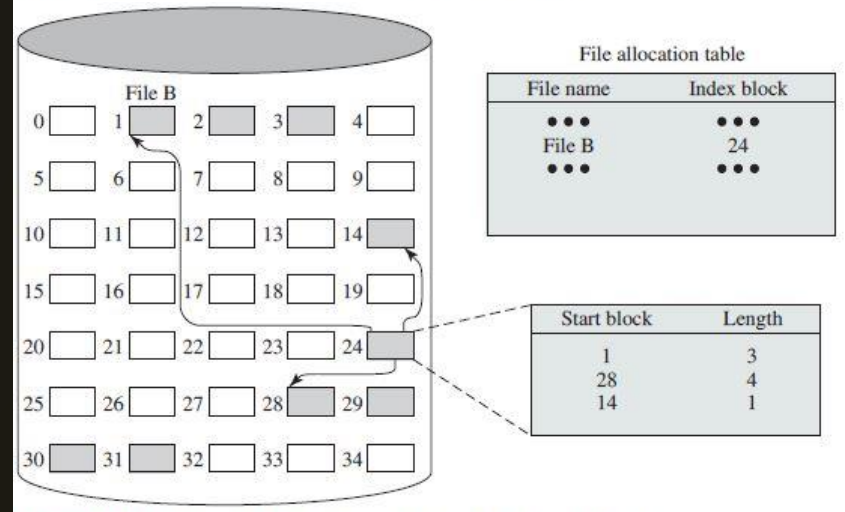



Figure 12.14 Indexed Allocation with Variable-Length Portions

# Free space management

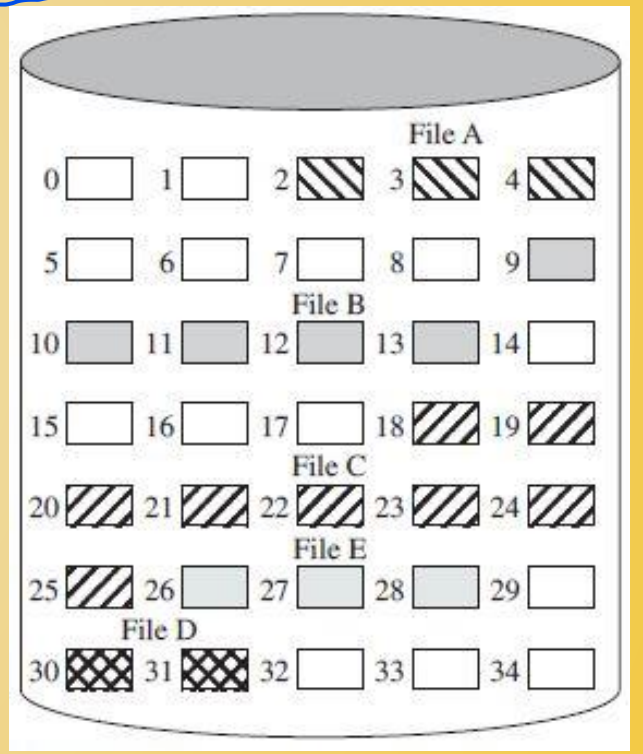
- Just as the space that is allocated to files must be managed, so the space that is not currently allocated to any file must be managed.
- To perform any of the file allocation techniques described earlier, it is necessary to know what blocks on the disk are available.
- We need a disk allocation table in addition to a file allocation table. Different ways of making such a table are:
  - Bit tables,
  - Chained free portions,
  - Indexing, and
  - Free block list.

disk allocation table : Keep track of which disk blocks are free and which are used

# Bit tables

Bit vector (35 bits):
00111000011111000011111111111011000

- This method uses a vector containing one bit for each block on the disk.
- Each entry of a 0 corresponds to a free block, and each 1 corresponds to a block in use.
- A bit table has the advantage that it is relatively easy to find one or a contiguous group of free blocks. Thus, a bit table works well with any of the file allocation methods outlined.
- Another advantage is that it is as small as possible.
- However, it can still be sizable.



# Bit table size

- The amount of memory (in bytes) required for a block bitmap is:

  disk size in bytes / ( 8 × file system block size )

- For a 16GB disk with 512-byte blocks, the bit table occupies 4MB.
- If it can be kept in main memory, then the bit table can be searched without the need for disk access.
- The alternative is to put the bit table on disk.
- If each block is of 512 bytes, or 0.5KB, a 4-Mbyte bit table requires 8,000 disk blocks.
- We can't afford to search that amount of disk space every time a block is needed.

# Optimizing bit tables

no separate deallocation table is needed

- Even when the bit table is resident, an exhaustive search of the table can slow file system performance to an unacceptable degree. This is especially true when the disk is nearly full and there are few free blocks remaining.
- Accordingly, most file systems that use bit tables maintain auxiliary data structures that summarize the contents of subranges of the bit table.
- The table is divided logically into a number of equal-size subranges. A summary table shows the number of free blocks and the largest free portion for each subrange. this is about bittables
- When the file system needs a portion, the summary table is scanned to find an appropriate subrange, which is then searched.

Even if the bitmap (bit table) is kept in RAM, the system may still need to scan many bits one by one to find a free block. This search becomes slow when the disk is almost full, because only a few free blocks exist and they may be scattered, most File systems use extra summary tables to quickly locate free space without scanning the entire bitmap

# Summarizing a bit table

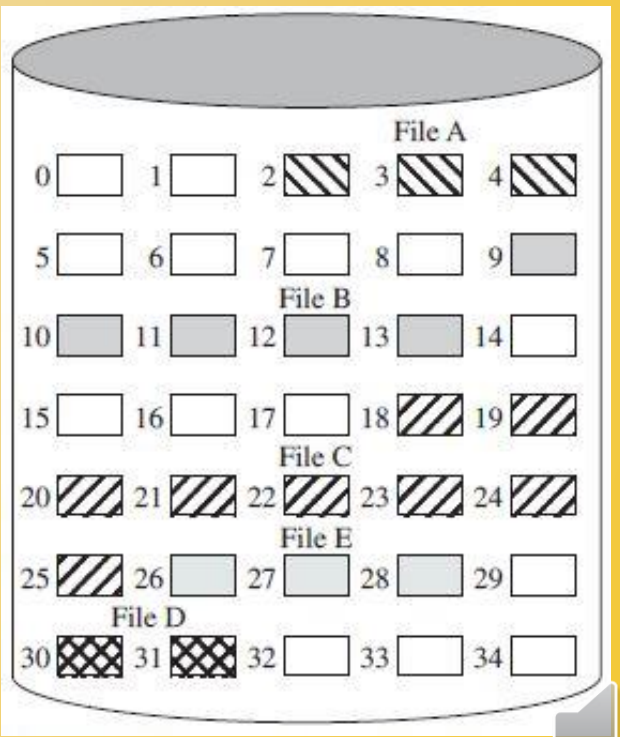
Summary table vs Bit table
Bit table (bitmap):
Main table that stores 1 bit per disk block (free/used status).
Summary table:
A separate auxiliary table used to speed up searching. It stores summary information for groups of bitmap entries (like number of free blocks, largest free chunk in that section)

this is a bit table

- Bit vector (35 bits): 00111000011111000011111111111011000
- Let us define a subrange to span 5 blocks.

summary table

| S. No. | Number of free blocks | Largest free portion size | Bit vector |
|---|---|---|---|
| 1 | 2 in total | 2 contagious | 00111 |
| 2 | 4 | 4 | 00001 |
| 3 | 1 | 1 | 11110 |
| 4 | 3 | 3 | 00011 |
| 5 | 0 | 0 | 11111 |
| 6 | 1 | 1 | 11110 |
| 7 | 3 | 3 | 11000 |



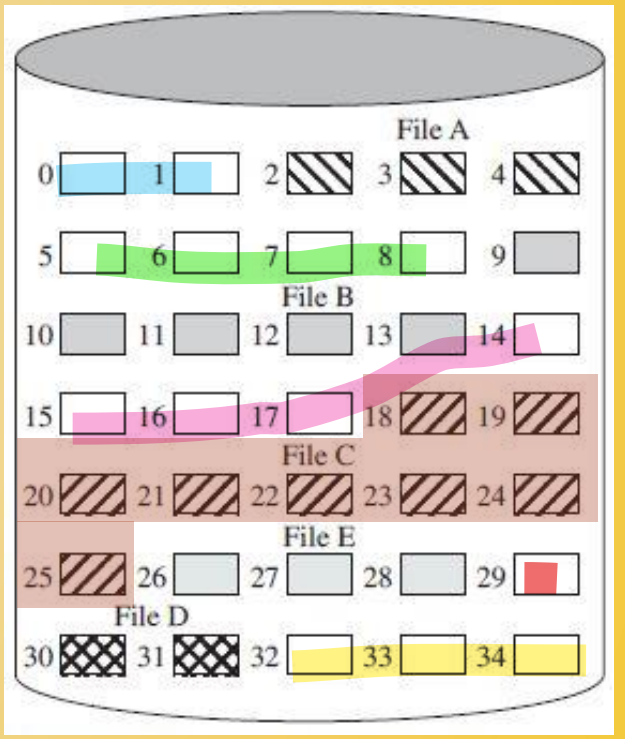
# Chained free portions

- The free portions may be chained together by using a pointer and length value in each free portion.
- This method has negligible space overhead because there is no need for a disk allocation table, merely for a pointer to the beginning of the chain and the length of the first portion.
- This method is suited to all of the file allocation methods.
- If allocation is a block at a time, simply choose the free block at the head of the chain and adjust the first pointer or length value.
- If allocation is by variable-length portion, a first-fit algorithm may be used: the headers from the portions are fetched one at a time to determine the next suitable free portion in the chain. Again, pointer and length values are adjusted.

# Creating a chain of free portions

- We keep a pointer to the first free portion, and its length, 0 (2).
- Each first block of a portion contains its length.
- The chain of free portions is:

  0 (2) → 5 (4) → 14 (4) → 29 (1) → 32 (3)
- If file C is deleted, we add its blocks to the chain:

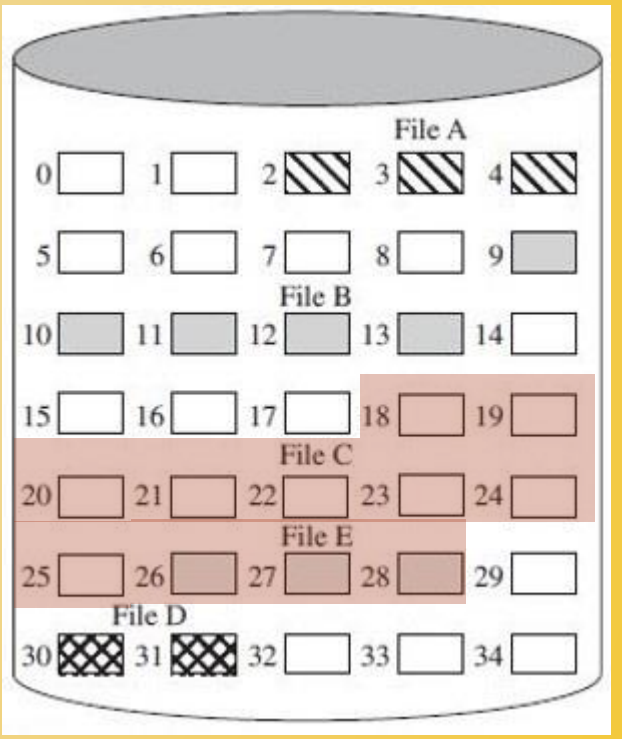


# Creating a chain of free portions

- We keep a pointer to the first free portion, and its length, 0 (2).
- Each first block of a portion contains its length.
- The chain of free portions is:

  0 (2) → 5 (4) → 14 (4) → 29 (1) → 32 (3)
- If file C is deleted, we add its blocks to the chain:

  0 (2) → 5 (4) → 14 (12) → 29 (1) → 32 (3)
- If file E is deleted, we add its blocks to the chain:



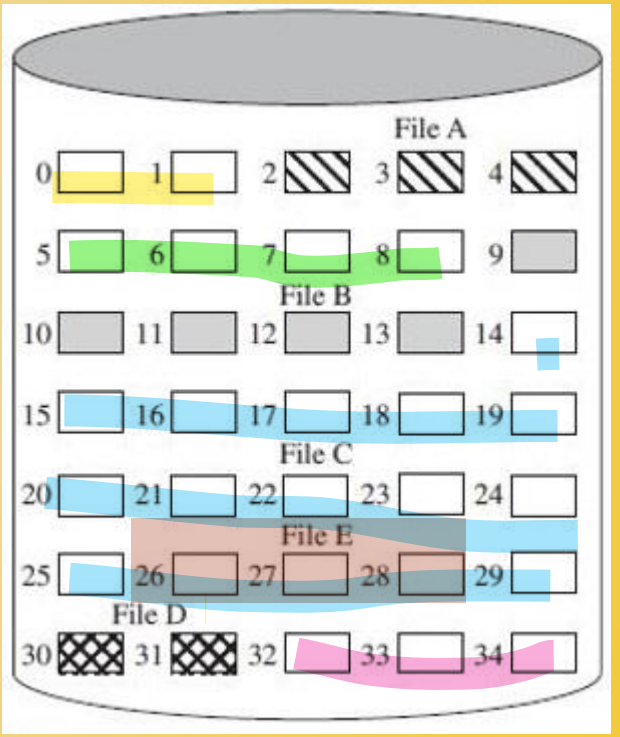
# Creating a chain of free portions

- We keep a pointer to the first free portion, and its length, 0 (2).
- Each first block of a portion contains its length.
- The chain of free portions is:

  0 (2) → 5 (4) → 14 (4) → 29 (1) → 32 (3)
- If file C is deleted, we add its blocks to the chain:

  0 (2) → 5 (4) → 14 (12) → 29 (1) → 32 (3)
- If file E is deleted, we add its blocks to the chain:

  0 (2) → 5 (4) → 14 (16) → 32 (3)
- We need a portion of 5 blocks. We use first fit:



# Creating a chain of free portions

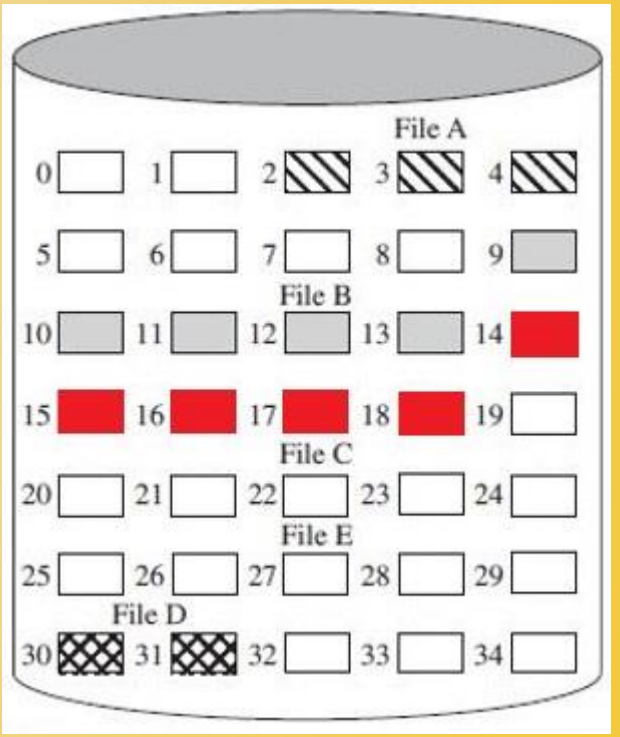
- We keep a pointer to the first free portion, and its length, 0 (2).
- Each first block of a portion contains its length.
- The chain of free portions is:

  0 (2) → 5 (4) → 14 (4) → 29 (1) → 32 (3)
- If file C is deleted, we add its blocks to the chain:

  0 (2) → 5 (4) → 14 (12) → 29 (1) → 32 (3)
- If file E is deleted, we add its blocks to the chain:

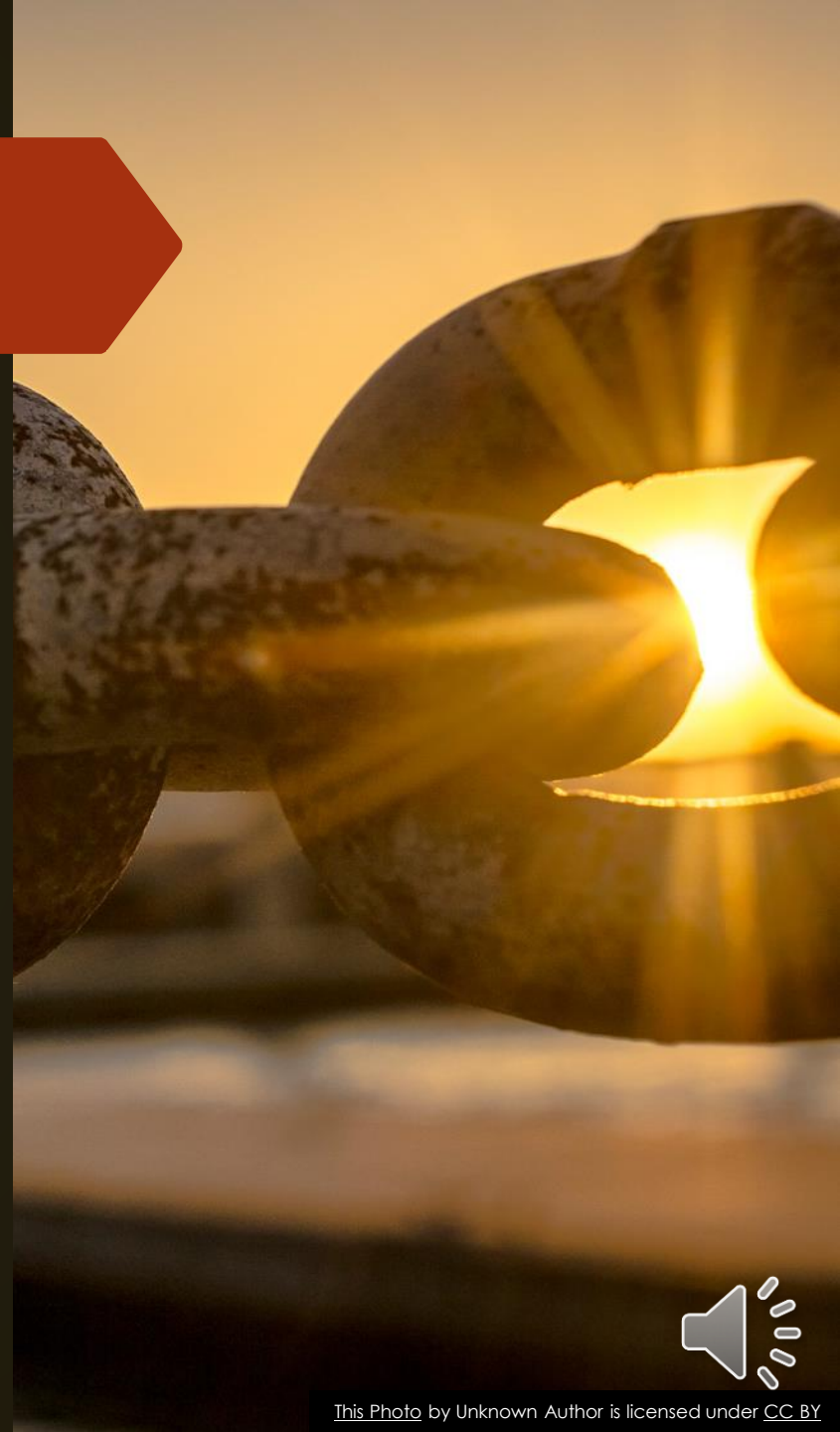
  0 (2) → 5 (4) → 14 (16) → 32 (3)
- We need a portion of 5 blocks. We use first fit:

  0 (2) → 5 (4) → 19 (11) → 32 (3)



# Consequences of chained free portions

- After some use, the disk will become quite fragmented and many portions will be a single block long.
- Every time you allocate a block, you need to read the block first to recover the pointer to the new first free block, before writing data to this block.
- If many individual blocks need to be allocated at one time for a file operation, this greatly slows file creation.
- Deleting highly fragmented files is very time consuming. We have to add each portion of the file to the existing chain of free blocks.



# Indexing

- The indexing approach treats free space as a file and uses an index table as described under indexed file allocation.
- For efficiency, the index should be on the basis of variable-sized portions rather than blocks.
- Thus, there is one entry in the table for every free portion on the disk.
- This approach provides efficient support for all of the file allocation methods.

# Free block list

- In this method, each block is assigned a number sequentially and the list of the numbers of all free blocks is maintained in a reserved portion of the disk.
- Depending on the size of the disk, either 24 or 32 bits will be needed to store a single block number.
- The size of the free block list is 24 or 32 times the size of the corresponding bit table and thus must be stored on disk rather than in main memory.

# Size of the free block list

- A 2 Terabytes disk has blocks of 512 bytes.
- Total number of blocks = 2TB / 0.5KB = $4 \times 2^{30} = 2^{32}$.
- Each block must be represented by a 32-bit or 4-byte number. The space penalty is 4 bytes for every 512-byte block.
- Total size of free block list = $4B \times 2^{32} = 16 \times 2^{30}$ B = 16GB.
- The free block list takes up less than 1% of the total disk space.

# Optimizing the free block list

- Although the free block list is too large to store in main memory, there are two effective techniques for storing a small part of the list in main memory:
  1. The list can be treated as a push-down stack with the first few thousand elements of the stack kept in main memory. When a new block is needed for allocation, it is popped from the top of the stack, which is in main memory. When a block is deallocated from a file, it is pushed onto the stack. A transfer between disk and main memory happens only when the in-memory portion of the stack becomes either empty (no free blocks) or full (the in-memory stack cannot receive more blocks). This technique gives almost zero-time access mostly.

# Optimizing the free block list

- Although the free block list is too large to store in main memory, there are two effective techniques for storing a small part of the list in main memory:
  2. The list can be treated as a FIFO queue, with a few thousand entries from both the head and the tail of the queue in main memory. A block is allocated by taking the first entry from the head of the queue and deallocated by adding it to the end of the tail of the queue. A transfer between disk and main memory happens only when either the in-memory portion of the head of the queue becomes empty (no free blocks) or the in-memory portion of the tail of the queue becomes full (the in-memory tail cannot receive more blocks).

# Volume

- A collection of addressable sectors in secondary memory that an OS or application can use for data storage is a volume.
- The sectors in a volume need not be consecutive on a physical storage device; instead, they need only appear that way to the OS or application.
- A volume may be the result of assembling and merging smaller volumes.



# More about volumes

- The term volume is used somewhat differently by different operating systems and file management systems, but in essence a volume is a logical disk.
- In the simplest case, a single disk equals one volume.
- Frequently, a disk is divided into partitions, with each partition functioning as a separate volume.
- It is also common to treat multiple disks as a single volume or partitions on multiple disks as a single volume.

# A file allocation gone wrong!

- Consider the following scenario:
  1. User A requests more space to add to an existing file.
  2. The request is granted and the disk and file allocation tables are updated in main memory but not yet on disk.
  3. The system crashes and subsequently restarts.
  4. User B requests more space and is allocated space on disk that overlaps the last allocation to user A.
  5. User A accesses the overlapped portion via a reference that is stored inside A's file. User A is inadvertently accessing user B's file.
- This difficulty arose because the system maintained a copy of the disk and file allocation tables in main memory for efficiency.

# Solution to the problem

- To prevent the type of error we just saw, we take the following steps:
  1. Lock the disk allocation table on disk. This prevents another user from altering the table until this allocation is complete.
  2. Search the disk allocation table for available space in the resident copy of the disk allocation table or read the table into main memory first and then search.
  3. Allocate space, update the disk allocation table, and update the disk. Updating the disk involves writing the disk allocation table back onto disk. For chained disk allocation, it also involves updating pointers.
  4. Update the file allocation table and update the disk.
  5. Unlock the disk allocation table.
- This technique will prevent errors.

# Reliable batch allocation

- When small portions are allocated frequently, performance suffers.
- A batch storage allocation scheme involves obtaining a batch of free portions from the disk for allocation.
- The corresponding portions on disk are marked in use.
- Allocation using this batch proceeds in main memory.
- When the batch is exhausted, the disk allocation table is updated on disk and a new batch is acquired.
- If a system crash occurs, portions on the disk marked in use must be cleaned up in some fashion before they can be reallocated. The technique for cleanup will depend on the file system's characteristics.

# File System Security

Protecting data

# Control of users' accesses

- Through the log-in name and password, a user is identified to the system. Each user has a profile specifying permissible operations and file accesses. The operating system can enforce rules based on the user profile.
- The database management system, however, must control access to specific records or even portions of records.
- Whereas the operating system grants a user permission to access a file or use an application, following which there are no further security checks, the database management system must make a decision on each individual access attempt.
- The DBMS' decision will depend not only on the user's identity but also on the specific parts of the data being accessed.

# Access matrix

- A general model of access control is an access matrix.
- The basic elements of the model are as follows:
  1. Subject: An entity capable of accessing objects. Generally, the subject is a process.
  2. Object: Anything to which access is controlled. Examples include files, portions of files, programs, segments of memory, and software objects (e.g., Java objects).
  3. Access right: The way in which an object is accessed by a subject. Examples are read, write, execute, and functions in software objects.

# Dimensions of the access matrix

- One dimension of the matrix consists of identified subjects. Typically, this list will consist of individual users or user groups, although access could be controlled for terminals, hosts, or applications instead of or in addition to users.
- The other dimension lists the objects. At the greatest level of detail, objects may be individual data fields. More aggregate groupings, such as records, files, or even the entire database, may also be objects in the matrix.
- Each entry in the matrix indicates the access rights of that subject for that object.
- In practice, an access matrix is usually sparse.

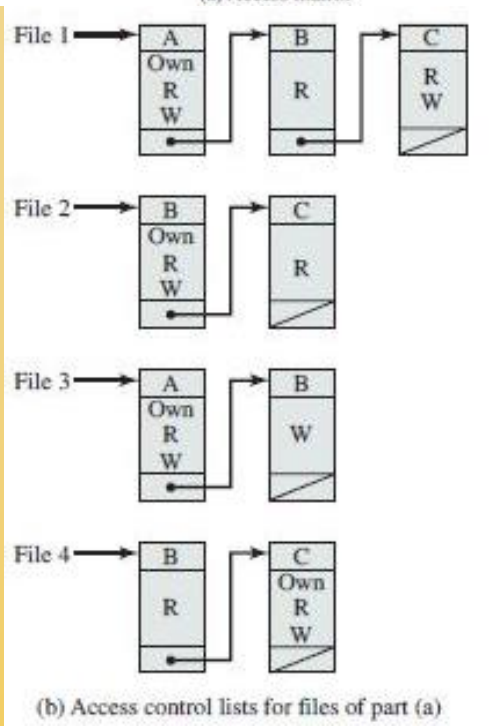
| | File 1 | File 2 | File 3 | File 4 | Account 1 | Account 2 |
|---|---|---|---|---|---|---|
| User A | Own<br>R<br>W | | Own<br>R<br>W | | Inquiry credit | |
| User B | R | Own<br>R<br>W | W | R | Inquiry debit | Inquiry credit |
| User C | R<br>W | R | | Own<br>R<br>W | | Inquiry debit |

# Access control list

- The matrix may be decomposed by columns, yielding access control lists.
- Thus for each object, an access control list lists users and their permitted access rights.
- The access control list may contain a default, or public, entry. This allows users that are not explicitly listed as having special rights to have a default set of rights.
- Elements of the list may include individual users as well as groups of users.

| | File 1 | File 2 | File 3 | File 4 | Account 1 | Account 2 |
|---|---|---|---|---|---|---|
| User A | Own<br>R<br>W | | Own<br>R<br>W | | Inquiry credit | |
| User B | R | Own<br>R<br>W | W | R | Inquiry debit | Inquiry credit |
| User C | R<br>W | R | | Own<br>R<br>W | | Inquiry debit |

(a) Access matrix



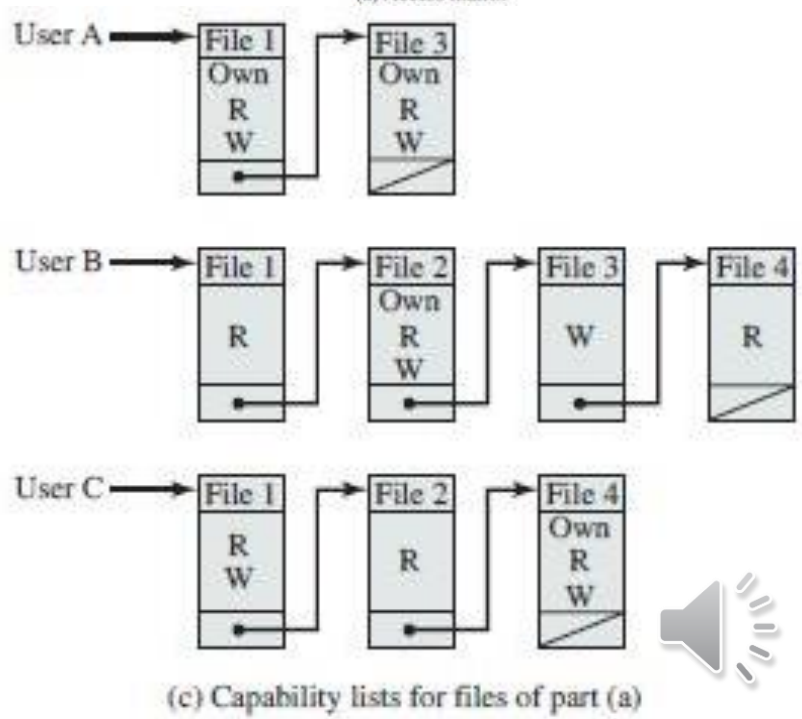

(b) Access control lists for files of part (a)

# Capability tickets

- Decomposition of access matrix by rows yields capability tickets.
- A capability ticket shows authorized objects and operations for a user. Each user has a number of tickets and may be authorized to loan or give them to others.
- Because tickets may be dispersed around the system, they present a greater security problem than access control lists.
- The ticket must be unforgeable. One way to accomplish this is to have the operating system hold all tickets on behalf of users. These tickets would have to be held in a region of memory inaccessible to users.

| | File 1 | File 2 | File 3 | File 4 | Account 1 | Account 2 |
|---|---|---|---|---|---|---|
| User A | Own<br>R<br>W | | Own<br>R<br>W | | Inquiry credit | |
| User B | R | Own<br>R<br>W | W | R | Inquiry debit | Inquiry credit |
| User C | R<br>W | R | | Own<br>R<br>W | | Inquiry debit |

(a) Access matrix




(c) Capability lists for files of part (a)

# What did we learn?

- We explored secondary storage management.
- We had a detailed discussion on file allocation methods.
- We talked about free space management.
- We understood the challenges of file system security.

# Things to do

| | |
|---|---|
| Read | Read the book. |
| Find | I will ask you to submit the word file previously saved as <my_roll_no>_lecture_23 in Google classroom. |
| Note | Write your questions on a piece of paper and keep it in front of you during the online session. |
| Email | If you have suggestions relevant to content or quality of this lecture, email them to me. |